

并查集算法专题

算法学习笔记

2026 年 5 月 8 日

目录

1	并查集基础	2
1.1	标准并查集（路径压缩 + 按秩合并）	2
1.2	带权并查集（维护到根节点的距离）	4
1.3	可持久化并查集	6
2	并查集扩展应用	10
2.1	连通块统计	10
2.2	并查集求连通分量个数	13
2.3	并查集判环（无向图）	15
2.4	并查集求图的连通块数量（固定图）	17
2.5	并查集求图的二分图判定（动态判断）	18
2.6	逆向并查集（离线倒序处理）	21
2.7	并查集求最小生成树（Kruskal 算法）	23
2.8	并查集求最近公共祖先（LCA）	25
3	算法复杂度总结	28

1 并查集基础

1.1 标准并查集（路径压缩 + 按秩合并）

解决问题：维护一个元素集合，支持两种操作：合并两个元素所在的集合、查询两个元素是否属于同一集合。适用于社交网络好友关系判断、连通分量统计、Kruskal 最小生成树等场景。

题目描述

给定 n 个元素（编号 $1 \sim n$ ），初始每个元素自成一个集合。有 m 次操作，操作类型有两种：1 $x \ y$ 表示合并 x 和 y 所在的集合；2 $x \ y$ 表示查询 x 和 y 是否在同一个集合中。

输入示例

```
5 6
1 1 2
1 2 3
2 1 3
1 4 5
2 1 4
2 4 5
```

输出示例

```
Yes No Yes
```

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      struct DSU {
4          vector<int> parent; // parent[i]: 节点i的父节点，根节点的父节点指向自己
5          vector<int> rank; // rank[i]: 节点i所在的树的高度（按秩合并时使用）
6          DSU(int n) { // 构造函数：初始化并查集
7              parent.resize(n + 1); // 索引从1到n，所以分配n+1个空间
8              rank.resize(n + 1, 0); // 初始所有节点高度为0
9              for (int i = 1; i <= n; i++) {
10                 parent[i] = i; // 初始时每个节点的父节点都是自己，即每个元素自成一个集合
11             }
12         }
```

```
13 // 核心原理：查找节点x所在集合的根节点（代表元），同时进行路径压缩优化
14 // 路径压缩的作用：将查找路径上的所有节点直接指向根节点，降低后续查找的
    时间复杂度
15 int find(int x) {
16     if (parent[x] != x) { // 如果x不是根节点
17         parent[x] = find(parent[x]); // 递归查找父节点，并将结果赋值给
            parent[x]（路径压缩）
18     }
19     return parent[x]; // 返回根节点
20 }
21 // 合并两个节点所在的集合
22 void unite(int x, int y) {
23     int rootX = find(x); // 找到x的根节点
24     int rootY = find(y); // 找到y的根节点
25     if (rootX == rootY) return; // 如果已经在同一集合，无需合并
26     // 按秩合并：将高度较小的树合并到高度较大的树，保持树平衡
27     if (rank[rootX] < rank[rootY]) {
28         parent[rootX] = rootY; // 将rootX的父节点设为rootY
29     } else if (rank[rootX] > rank[rootY]) {
30         parent[rootY] = rootX; // 将rootY的父节点设为rootX
31     } else {
32         parent[rootY] = rootX; // 高度相等时，任意合并，并将高度+1
33         rank[rootX]++; // 新树的高度增加1
34     }
35 }
36 // 查询两个节点是否在同一个集合中
37 bool same(int x, int y) {
38     return find(x) == find(y);
39 }
40 };
41 int main() {
42     int n, m; // n: 元素总数, m: 操作次数
43     cin >> n >> m;
44     DSU dsu(n); // 创建大小为n的并查集
45     for (int i = 0; i < m; i++) {
46         int op, x, y; // op: 操作类型, x, y: 操作的两个元素
47         cin >> op >> x >> y;
48         if (op == 1) {
49             dsu.unite(x, y); // 合并操作
50         } else {
```

```

51         if (dsu.same(x, y)) { // 查询操作
52             cout << "Yes\n";
53         } else {
54             cout << "No\n";
55         }
56     }
57 }
58 return 0;
59 }

```

1.2 带权并查集（维护到根节点的距离）

解决问题：维护元素之间的相对关系，如食物链中的捕食关系、向量偏移、区间和关系等。适用于解决带关系的集合合并问题，每个元素到根节点的距离有意义。

题目描述

有 n 个动物（编号 $1 \sim n$ ），被分成三类：A 吃 B，B 吃 C，C 吃 A。给出 k 句话，每句话描述 x 和 y 的关系（1 表示 x 和 y 同类，2 表示 x 吃 y ）。判断假话的数量（假话条件：与前面的话冲突，或者编号超出范围）。

输入示例

```

100 7
1 101 1
2 1 2
2 2 3
2 3 3
1 1 3
2 3 1
1 5 5

```

输出示例

3

```

1     #include <bits/stdc++.h>
2     using namespace std;
3     struct WeightedDSU {
4         vector<int> parent; // parent[i]: 节点i的父节点

```

```

5     vector<int> dist; // dist[i]: 节点i到其父节点的关系 (0表示同类, 1表示
        被父节点吃, 2表示吃父节点)
6     WeightedDSU(int n) { // 构造函数: 初始化带权并查集
7         parent.resize(n + 1);
8         dist.resize(n + 1, 0); // 初始到父节点的距离为0 (自己到自己是同类)
9         for (int i = 1; i <= n; i++) {
10             parent[i] = i; // 初始每个节点自成一棵树
11         }
12     }
13     // 核心原理: 查找根节点时, 同时更新dist[x]为x到根节点的关系
14     // 关系通过路径上的关系值相加并取模3得到 (因为只有3种关系)
15     int find(int x) {
16         if (parent[x] != x) {
17             int root = find(parent[x]); // 递归找到根节点
18             dist[x] = (dist[x] + dist[parent[x]]) % 3; // 更新x到根节点的关系
19             parent[x] = root; // 路径压缩, 直接指向根节点
20         }
21         return parent[x];
22     }
23     // 合并操作: 根据给定关系type合并x和y
24     // type=0表示x和y同类, type=1表示x吃y
25     bool unite(int x, int y, int type) {
26         int rootX = find(x); // 找到x的根节点
27         int rootY = find(y); // 找到y的根节点
28         if (rootX == rootY) { // 已经同根, 检查是否与已有关系矛盾
29             int relation = (dist[x] - dist[y] + 3) % 3; // 计算x到y的关系
30             return relation == type; // 如果计算出的关系与给定的type一致,
                则真话; 否则假话
31         }
32         // 合并两个树: 将rootX合并到rootY, 并更新dist[rootX]
33         parent[rootX] = rootY;
34         // 推导dist[rootX]的公式: 已知dist[x]到rootX, dist[y]到rootY
35         // 需要满足: x到y的关系 = (dist[x] + dist[rootX] - dist[y]) % 3 =
            type
36         // 可得: dist[rootX] = (type + dist[y] - dist[x]) % 3
37         dist[rootX] = (type + dist[y] - dist[x] + 3) % 3;
38         return true; // 合并成功, 这句话是真话
39     }
40 };

```

```
41     int main() {
42         int n, k; // n: 动物总数, k: 话的句数
43         cin >> n >> k;
44         WeightedDSU dsu(n); // 创建大小为n的带权并查集
45         int falseCount = 0; // falseCount: 假话的数量
46         for (int i = 0; i < k; i++) {
47             int d, x, y; // d: 关系类型 (1或2), x, y: 话中涉及的两个动物编号
48             cin >> d >> x >> y;
49             if (x > n || y > n) { // 条件2: 编号超出范围, 直接判为假话
50                 falseCount++;
51                 continue;
52             }
53             int type = d - 1; // 将1/2转换为0/1, 0表示同类, 1表示x吃y
54             if (!dsu.unite(x, y, type)) { // 如果与已有关系矛盾
55                 falseCount++;
56             }
57         }
58         cout << falseCount << "\n";
59         return 0;
60     }
```

1.3 可持久化并查集

解决问题：维护并查集的历史版本，支持回到之前的版本查询。适用于需要回滚操作的并查集问题、版本控制、时间旅行等场景。

题目描述

给定 n 个元素，初始每个元素自成一个集合。有 m 个操作，操作类型：1 x y 合并 x 和 y 所在的集合；2 k 回到第 k 次操作后的状态（操作编号从 1 开始）；3 x y 查询 x 和 y 是否在同一集合中。

输入示例

```
5 8
1 1 2
1 2 3
3 1 3
2 1
3 1 3
```

2 2
3 1 3
3 2 3

输出示例

Yes No Yes Yes

```

1      #include <bits/stdc++.h>
2      using namespace std;
3      struct PersistentDSU {
4          struct Node {
5              int parent; // parent: 父节点编号
6              int rank;   // rank: 树的高度（按秩合并用）
7              int left;   // left: 左子节点索引（主席树左儿子）
8              int right;  // right: 右子节点索引（主席树右儿子）
9          };
10         vector<Node> tree; // tree: 主席树节点数组
11         vector<int> roots; // roots[i]: 第i个版本的根节点索引
12         int n; // n: 元素数量
13         int nodeCount; // nodeCount: 当前已创建的节点数量
14         PersistentDSU(int n) : n(n), nodeCount(0) {
15             tree.resize(n * 60); // 预分配足够空间，一般n*60足够
16             roots.push_back(build(1, n)); // 版本0: 初始状态
17         }
18         // 构建初始线段树（叶子节点存储每个元素的父节点和rank）
19         int build(int l, int r) {
20             int idx = ++nodeCount; // 分配新节点
21             if (l == r) {
22                 tree[idx].parent = l; // 初始父节点为自己
23                 tree[idx].rank = 0; // 初始高度为0
24                 return idx;
25             }
26             int mid = (l + r) / 2;
27             tree[idx].left = build(l, mid); // 递归构建左子树
28             tree[idx].right = build(mid + 1, r); // 递归构建右子树
29             return idx;
30         }
31         // 查询某个版本中位置pos的节点信息（父节点和rank）
32         pair<int, int> query(int root, int l, int r, int pos) {

```

```

33     if (l == r) {
34         return {tree[root].parent, tree[root].rank};
35     }
36     int mid = (l + r) / 2;
37     if (pos <= mid) {
38         return query(tree[root].left, l, mid, pos);
39     } else {
40         return query(tree[root].right, mid + 1, r, pos);
41     }
42 }
43 // 更新某个版本中位置pos的父节点值
44 int update(int root, int l, int r, int pos, int newParent, int
45     newRank) {
46     int idx = ++nodeCount; // 创建新节点（可持久化的关键：修改时新建节
47         点）
48     tree[idx] = tree[root]; // 复制原节点的左右孩子
49     if (l == r) {
50         tree[idx].parent = newParent; // 更新父节点
51         tree[idx].rank = newRank; // 更新rank
52         return idx;
53     }
54     int mid = (l + r) / 2;
55     if (pos <= mid) {
56         tree[idx].left = update(tree[root].left, l, mid, pos,
57             newParent, newRank);
58     } else {
59         tree[idx].right = update(tree[root].right, mid + 1, r, pos,
60             newParent, newRank);
61     }
62     return idx;
63 }
64 // 查找节点x的根节点（在指定版本中）
65 int find(int version, int x) {
66     auto [parent, rank] = query(roots[version], 1, n, x);
67     if (parent == x) return x;
68     return find(version, parent); // 注意：可持久化并查集不能做路径压缩
69         （会破坏历史版本）
70 }
71 // 合并操作，返回新版本的根节点索引
72 int unite(int version, int x, int y) {

```



```

68     int rootX = find(version, x);
69     int rootY = find(version, y);
70     if (rootX == rootY) return roots[version]; // 已经在同一集合, 返回
        原版本
71     // 查询两个根节点的rank
72     auto [px, rankX] = query(roots[version], 1, n, rootX);
73     auto [py, rankY] = query(roots[version], 1, n, rootY);
74     int newRoot;
75     // 按秩合并: 将高度小的树合并到高度大的树
76     if (rankX < rankY) {
77         // 将rootX的父节点设为rootY
78         newRoot = update(roots[version], 1, n, rootX, rootY, rankX);
79     } else if (rankX > rankY) {
80         // 将rootY的父节点设为rootX
81         newRoot = update(roots[version], 1, n, rootY, rootX, rankY);
82     } else {
83         // 高度相等, 将rootY合并到rootX, 并将rootX的rank+1
84         newRoot = update(roots[version], 1, n, rootY, rootX, rankY);
85         newRoot = update(newRoot, 1, n, rootX, rootX, rankX + 1);
86     }
87     return newRoot;
88 }
89 // 保存新版本
90 void addVersion(int root) {
91     roots.push_back(root);
92 }
93 // 查询两个节点是否在同一集合 (指定版本)
94 bool same(int version, int x, int y) {
95     return find(version, x) == find(version, y);
96 }
97 };
98 int main() {
99     int n, m; // n: 元素个数, m: 操作次数
100     cin >> n >> m;
101     PersistentDSU dsu(n); // 创建可持久化并查集
102     for (int i = 0; i < m; i++) {
103         int op; // op: 操作类型
104         cin >> op;
105         if (op == 1) { // 合并操作
106             int x, y; // x, y: 要合并的两个元素

```

```
107         cin >> x >> y;
108         int newRoot = dsu.unite(dsu.roots.size() - 1, x, y);
109         dsu.addVersion(newRoot);
110     } else if (op == 2) { // 回滚操作
111         int k; // k: 回到第k次操作后的版本
112         cin >> k;
113         dsu.addVersion(dsu.roots[k]); // 复制第k个版本作为当前版本
114     } else if (op == 3) { // 查询操作
115         int x, y; // x, y: 要查询的两个元素
116         cin >> x >> y;
117         bool result = dsu.same(dsu.roots.size() - 1, x, y);
118         cout << (result ? "Yes" : "No") << "\n";
119         dsu.addVersion(dsu.roots.back()); // 查询也新建版本（保持版本数
            一致）
120     }
121 }
122 return 0;
123 }
```

2 并查集扩展应用

2.1 连通块统计

解决问题：维护并查集的同时统计每个集合的大小（元素个数）。适用于动态连通性问题中需要获取集合大小、朋友圈人数统计等场景。

题目描述

有 n 个人，初始相互不认识。每次操作添加一条朋友关系，或者查询某个人所在的朋友圈有多少人，或者查询当前最大的朋友圈有多少人。

输入示例

```
10 8
1 1 2
2 1
1 3 4
1 2 3
2 1
3
```

1 5 6

3

输出示例

2 4 4 2

```

1      #include <bits/stdc++.h>
2      using namespace std;
3      struct DSUWithSize {
4          vector<int> parent; // parent[i]: 节点i的父节点
5          vector<int> size; // size[i]: 节点i所在集合的元素个数 (仅根节点有效)
6          vector<int> rank; // rank[i]: 树的高度 (用于按秩合并)
7          int maxSize; // maxSize: 当前最大集合的大小
8          DSUWithSize(int n) { // 构造函数: 初始化
9              parent.resize(n + 1);
10             size.resize(n + 1, 1); // 初始每个集合大小为1
11             rank.resize(n + 1, 0); // 初始高度为0
12             maxSize = 1; // 初始最大集合大小为1
13             for (int i = 1; i <= n; i++) {
14                 parent[i] = i; // 初始父节点指向自己
15             }
16         }
17         int find(int x) { // 查找根节点 (带路径压缩)
18             if (parent[x] != x) {
19                 parent[x] = find(parent[x]); // 路径压缩
20             }
21             return parent[x];
22         }
23         // 合并两个元素所在的集合
24         void unite(int x, int y) {
25             int rootX = find(x);
26             int rootY = find(y);
27             if (rootX == rootY) return; // 已经是同一集合
28             // 按秩合并: 将高度小的树合并到高度大的树
29             if (rank[rootX] < rank[rootY]) {
30                 parent[rootX] = rootY;
31                 size[rootY] += size[rootX]; // 集合大小累加
32                 maxSize = max(maxSize, size[rootY]); // 更新最大集合大小
33             } else if (rank[rootX] > rank[rootY]) {

```

```

34         parent[rootY] = rootX;
35         size[rootX] += size[rootY];
36         maxSize = max(maxSize, size[rootX]);
37     } else {
38         parent[rootY] = rootX;
39         size[rootX] += size[rootY];
40         rank[rootX]++; // 高度相等，新树高度+1
41         maxSize = max(maxSize, size[rootX]);
42     }
43 }
44 // 查询x所在集合的大小
45 int getSize(int x) {
46     return size[find(x)];
47 }
48 // 查询当前最大集合的大小
49 int getMaxSize() {
50     return maxSize;
51 }
52 };
53 int main() {
54     int n, m; // n: 人数, m: 操作次数
55     cin >> n >> m;
56     DSUWithSize dsu(n); // 创建带大小统计的并查集
57     for (int i = 0; i < m; i++) {
58         int op; // op: 操作类型
59         cin >> op;
60         if (op == 1) { // 添加朋友关系 (合并)
61             int x, y; // x, y: 成为朋友的两个人编号
62             cin >> x >> y;
63             dsu.unite(x, y);
64         } else if (op == 2) { // 查询某人的朋友圈大小
65             int x; // x: 要查询的人编号
66             cin >> x;
67             cout << dsu.getSize(x) << "\n";
68         } else if (op == 3) { // 查询当前最大朋友圈大小
69             cout << dsu.getMaxSize() << "\n";
70         }
71     }
72     return 0;
73 }

```

2.2 并查集求连通分量个数

解决问题：在一个不断添加边或者初始固定的图中，动态维护连通分量的数量。适用于网络连通性分析、岛屿数量统计等场景。

题目描述

有 n 个节点（编号 $1 \sim n$ ），初始没有边。每次操作添加一条边，并输出当前连通分量的数量。

输入示例

```
5 4
1 2
2 3
1 3
4 5
```

输出示例

```
4 3 3 2
```

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      struct DSUWithCount {
4          vector<int> parent; // parent[i]: 节点i的父节点
5          vector<int> rank; // rank[i]: 树的高度
6          int components; // components: 当前连通分量的数量
7          DSUWithCount(int n) { // 构造函数: 初始化n个节点
8              parent.resize(n + 1);
9              rank.resize(n + 1, 0);
10             components = n; // 初始每个节点自成一个连通分量, 共n个
11             for (int i = 1; i <= n; i++) {
12                 parent[i] = i;
13             }
14         }
15         int find(int x) { // 查找根节点 (带路径压缩)
16             if (parent[x] != x) {
17                 parent[x] = find(parent[x]);
```

```
18     }
19     return parent[x];
20 }
21 // 合并两个节点所在的集合, 返回是否成功合并 (即是否减少了连通分量数量)
22 bool unite(int x, int y) {
23     int rootX = find(x);
24     int rootY = find(y);
25     if (rootX == rootY) return false; // 已经在同一集合, 未减少分量
26     // 按秩合并
27     if (rank[rootX] < rank[rootY]) {
28         parent[rootX] = rootY;
29     } else if (rank[rootX] > rank[rootY]) {
30         parent[rootY] = rootX;
31     } else {
32         parent[rootY] = rootX;
33         rank[rootX]++;
34     }
35     components--; // 成功合并, 连通分量数量减1
36     return true;
37 }
38 int getComponents() { // 获取当前连通分量数量
39     return components;
40 }
41 };
42 int main() {
43     int n, m; // n: 节点总数, m: 边的数量
44     cin >> n >> m;
45     DSUWithCount dsu(n); // 创建连通分量计数的并查集
46     for (int i = 0; i < m; i++) {
47         int u, v; // u, v: 添加的边连接的两个节点
48         cin >> u >> v;
49         dsu.unite(u, v);
50         cout << dsu.getComponents() << "\n";
51     }
52     return 0;
53 }
```

2.3 并查集判环（无向图）

解决问题：在动态加边的过程中判断是否形成环。适用于检测无向图中的环、判断添加边后是否产生回路等场景。

题目描述

有 n 个节点，依次添加 m 条边。每添加一条边，判断这条边是否会形成环（即边的两个端点是否已经连通）。

输入示例

```
5 5
1 2
2 3
3 4
4 5
1 5
```

输出示例

```
No No No No Yes
```

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      struct DSUCycle {
4          vector<int> parent; // parent[i]: 节点i的父节点
5          vector<int> rank; // rank[i]: 树的高度
6          DSUCycle(int n) { // 构造函数: 初始化n个节点
7              parent.resize(n + 1);
8              rank.resize(n + 1, 0);
9              for (int i = 1; i <= n; i++) {
10                 parent[i] = i; // 初始父节点指向自己
11             }
12         }
13         int find(int x) { // 查找根节点（带路径压缩）
14             if (parent[x] != x) {
15                 parent[x] = find(parent[x]);
16             }
17             return parent[x];
18         }
19     }
```

```
19 // 尝试合并两个节点, 如果已经在同一集合则说明加边后会形成环
20 // 返回值: true表示会形成环, false表示不会形成环
21 bool addEdge(int x, int y) {
22     int rootX = find(x);
23     int rootY = find(y);
24     if (rootX == rootY) {
25         return true; // 已经连通, 添加这条边会产生环
26     }
27     // 按秩合并
28     if (rank[rootX] < rank[rootY]) {
29         parent[rootX] = rootY;
30     } else if (rank[rootX] > rank[rootY]) {
31         parent[rootY] = rootX;
32     } else {
33         parent[rootY] = rootX;
34         rank[rootX]++;
35     }
36     return false; // 成功合并, 不会产生环
37 }
38 };
39 int main() {
40     int n, m; // n: 节点总数, m: 边的数量
41     cin >> n >> m;
42     DSUCycle dsu(n); // 创建判环并查集
43     for (int i = 0; i < m; i++) {
44         int u, v; // u, v: 添加的边连接的两个节点
45         cin >> u >> v;
46         if (dsu.addEdge(u, v)) {
47             cout << "Yes\n"; // 会形成环
48         } else {
49             cout << "No\n"; // 不会形成环
50         }
51     }
52     return 0;
53 }
```


2.4 并查集求图的连通块数量（固定图）

解决问题：给定一个固定的图，统计有多少个连通分量。适用于社交网络分析、图像连通区域标记等场景。

题目描述

给定 n 个节点和 m 条边，输出图中连通分量的个数。

输入示例

```
6 4
1 2
2 3
4 5
5 6
```

输出示例

```
2
```

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  int find(vector<int>& parent, int x) { // 查找根节点（带路径压缩）
4      if (parent[x] != x) {
5          parent[x] = find(parent, parent[x]);
6      }
7      return parent[x];
8  }
9  void unite(vector<int>& parent, vector<int>& rank, int x, int y) { // 合并两个节点
10     int rootX = find(parent, x);
11     int rootY = find(parent, y);
12     if (rootX == rootY) return;
13     if (rank[rootX] < rank[rootY]) {
14         parent[rootX] = rootY;
15     } else if (rank[rootX] > rank[rootY]) {
16         parent[rootY] = rootX;
17     } else {
18         parent[rootY] = rootX;
19         rank[rootX]++;
20     }
```

```
20     }
21 }
22 int main() {
23     int n, m; // n: 节点总数, m: 边的数量
24     cin >> n >> m;
25     vector<int> parent(n + 1); // parent[i]: 节点i的父节点
26     vector<int> rank(n + 1, 0); // rank[i]: 树的高度
27     for (int i = 1; i <= n; i++) {
28         parent[i] = i; // 初始父节点指向自己
29     }
30     for (int i = 0; i < m; i++) {
31         int u, v; // u, v: 边连接的两个节点
32         cin >> u >> v;
33         unite(parent, rank, u, v); // 合并两个节点所在的连通块
34     }
35     int components = 0; // components: 连通分量数量
36     for (int i = 1; i <= n; i++) {
37         if (find(parent, i) == i) { // 根节点的数量就是连通分量的数量
38             components++;
39         }
40     }
41     cout << components << "\n";
42     return 0;
43 }
```

2.5 并查集求图的二分图判定（动态判断）

解决问题：在动态加边的过程中判断图是否为二分图。适用于染色问题、社交网络中的阵营划分等场景。

题目描述

有 n 个节点，初始没有边。每次操作添加一条边，判断当前图是否为二分图。

输入示例

```
4 5
1 2
2 3
3 4
```

1 4

1 3

输出示例

Yes Yes Yes No No

```

1      #include <bits/stdc++.h>
2      using namespace std;
3      struct DSUBipartite {
4          vector<int> parent; // parent[i]: 节点i的父节点
5          vector<int> rank; // rank[i]: 树的高度
6          vector<int> color; // color[i]: 节点i相对于根节点的颜色 (0表示同色, 1
              表示异色)
7          bool isBipartite; // isBipartite: 当前图是否为二分图
8          DSUBipartite(int n) { // 构造函数: 初始化n个节点
9              parent.resize(n + 1);
10             rank.resize(n + 1, 0);
11             color.resize(n + 1, 0); // 初始颜色为0
12             isBipartite = true; // 初始空图是二分图
13             for (int i = 1; i <= n; i++) {
14                 parent[i] = i; // 初始父节点指向自己
15             }
16         }
17         // 核心原理: 查找根节点, 同时更新节点到根节点的颜色关系
18         // 颜色用0/1表示, 路径上的颜色异或值即为节点到根节点的关系
19         int find(int x) {
20             if (parent[x] != x) {
21                 int root = find(parent[x]);
22                 color[x] ^= color[parent[x]]; // 更新x到根节点的颜色 (异或)
23                 parent[x] = root;
24             }
25             return parent[x];
26         }
27         // 添加边x-y, 如果已知是二分图则判断是否仍满足二分图性质
28         void addEdge(int x, int y) {
29             if (!isBipartite) return; // 已经不是二分图, 无需继续判断
30             int rootX = find(x);
31             int rootY = find(y);
32             if (rootX == rootY) {

```

```
33         // 两个节点已经在同一集合，检查颜色是否冲突
34         if (color[x] == color[y]) { // 同一集合且颜色相同，产生奇数环
35             isBipartite = false;
36         }
37     } else {
38         // 合并两个集合，根据边的关系确定根节点的颜色
39         // 边x-y表示x和y颜色不同: color[y] = color[x] ^ 1
40         // 需要确定rootY相对于rootX的颜色
41         if (rank[rootX] < rank[rootY]) {
42             parent[rootX] = rootY;
43             // 推导公式: color[rootX] = color[x] ^ color[y] ^ 1
44             color[rootX] = color[x] ^ color[y] ^ 1;
45         } else {
46             parent[rootY] = rootX;
47             color[rootY] = color[x] ^ color[y] ^ 1;
48             if (rank[rootX] == rank[rootY]) {
49                 rank[rootX]++;
50             }
51         }
52     }
53 }
54 bool check() { // 返回当前图是否为二分图
55     return isBipartite;
56 }
57 };
58 int main() {
59     int n, m; // n: 节点总数, m: 边的数量
60     cin >> n >> m;
61     DSUBipartite dsu(n); // 创建二分图判定并查集
62     for (int i = 0; i < m; i++) {
63         int u, v; // u, v: 添加的边连接的两个节点
64         cin >> u >> v;
65         dsu.addEdge(u, v);
66         cout << (dsu.check() ? "Yes" : "No") << "\n";
67     }
68     return 0;
69 }
```

2.6 逆向并查集（离线倒序处理）

解决问题：处理删除节点或删除边的并查集问题。由于并查集不支持删除操作，可以将操作离线反转，将删除变成添加操作。适用于图论的删边、删点问题。

题目描述

有 n 个节点（编号 $1 \sim n$ ）和 m 条边。随后有 q 个操作，每个操作删除一个节点。每步操作后，输出当前图中连通分量的数量。

输入示例

```
6 5
1 2
2 3
1 3
4 5
5 6
3
2
4
5
```

输出示例

```
2 3 4 5
```

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      struct DSU {
4          vector<int> parent; // parent[i]: 节点i的父节点
5          vector<int> rank; // rank[i]: 树的高度
6          int components; // components: 当前连通分量数量
7          DSU(int n) { // 构造函数: 初始化n个节点
8              parent.resize(n + 1);
9              rank.resize(n + 1, 0);
10             components = n;
11             for (int i = 1; i <= n; i++) {
12                 parent[i] = i;
13             }
14         }
```

```

15     int find(int x) { // 查找根节点（带路径压缩）
16         if (parent[x] != x) {
17             parent[x] = find(parent[x]);
18         }
19         return parent[x];
20     }
21     bool unite(int x, int y) { // 合并两个节点
22         int rootX = find(x);
23         int rootY = find(y);
24         if (rootX == rootY) return false;
25         if (rank[rootX] < rank[rootY]) {
26             parent[rootX] = rootY;
27         } else if (rank[rootX] > rank[rootY]) {
28             parent[rootY] = rootX;
29         } else {
30             parent[rootY] = rootX;
31             rank[rootX]++;
32         }
33         components--; // 成功合并，连通分量减1
34         return true;
35     }
36     int getComponents() { return components; }
37 };
38 int main() {
39     int n, m; // n: 节点总数, m: 边的数量
40     cin >> n >> m;
41     vector<pair<int, int>> edges(m); // edges: 存储所有边
42     for (int i = 0; i < m; i++) {
43         cin >> edges[i].first >> edges[i].second; // 读入每条边
44     }
45     int q; // q: 删除操作的数量
46     cin >> q;
47     vector<int> delNodes(q); // delNodes: 记录所有要删除的节点
48     vector<bool> isDeleted(n + 1, false); // isDeleted[i]: 标记节点i是否
        被删除
49     for (int i = 0; i < q; i++) {
50         cin >> delNodes[i];
51         isDeleted[delNodes[i]] = true; // 标记为已删除
52     }
53     // 核心原理：逆向处理，先假设所有要删除的节点都已删除，然后反向添加节点

```

```

54     DSU dsu(n);
55     // 先把所有不会被删除的边加入并查集
56     for (int i = 0; i < m; i++) {
57         int u = edges[i].first, v = edges[i].second;
58         if (!isDeleted[u] && !isDeleted[v]) { // 两个端点都没被删除才加入
59             dsu.unite(u, v);
60         }
61     }
62     vector<int> ans(q); // ans: 存储每一步的答案
63     // 逆序处理删除操作（实际上是在反向添加节点）
64     for (int i = q - 1; i >= 0; i--) {
65         ans[i] = dsu.getComponents(); // 记录当前状态的连通分量数
66         int node = delNodes[i]; // 当前正在添加的节点
67         isDeleted[node] = false; // 标记为未删除
68         // 扫描所有与node相连的边，将node与未删除的邻居合并
69         for (int j = 0; j < m; j++) {
70             int u = edges[j].first, v = edges[j].second;
71             if (u == node && !isDeleted[v]) {
72                 dsu.unite(u, v);
73             }
74             if (v == node && !isDeleted[u]) {
75                 dsu.unite(v, u);
76             }
77         }
78     }
79     for (int i = 0; i < q; i++) {
80         cout << ans[i] << "\n";
81     }
82     return 0;
83 }

```

2.7 并查集求最小生成树（Kruskal 算法）

解决问题：给定一个带权无向图，求最小生成树的总权值。适用于网络建设、道路规划、电路布线等场景。

题目描述

有 n 个节点和 m 条边，每条边有权值 w 。求最小生成树的总权值（如果图不连通则输出-1）。

输入示例

```
4 5
1 2 1
1 3 2
2 3 1
2 4 3
3 4 2
```

输出示例

```
4
```

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      struct Edge {
4          int u, v, w; // u: 边的起点, v: 边的终点, w: 边的权值
5          Edge(int u, int v, int w) : u(u), v(v), w(w) {}
6          bool operator<(const Edge& other) const {
7              return w < other.w; // 按权值升序排序
8          }
9      };
10     struct DSU {
11         vector<int> parent; // parent[i]: 节点i的父节点
12         vector<int> rank; // rank[i]: 树的高度
13         DSU(int n) {
14             parent.resize(n + 1);
15             rank.resize(n + 1, 0);
16             for (int i = 1; i <= n; i++) parent[i] = i;
17         }
18         int find(int x) {
19             if (parent[x] != x) parent[x] = find(parent[x]);
20             return parent[x];
21         }
22         bool unite(int x, int y) {
23             int rootX = find(x), rootY = find(y);
24             if (rootX == rootY) return false;
25             if (rank[rootX] < rank[rootY]) parent[rootX] = rootY;
26             else if (rank[rootX] > rank[rootY]) parent[rootY] = rootX;
27             else { parent[rootY] = rootX; rank[rootX]++; }
```



```
28         return true;
29     }
30 };
31 int main() {
32     int n, m; // n: 节点总数, m: 边的总数
33     cin >> n >> m;
34     vector<Edge> edges; // edges: 存储所有边
35     for (int i = 0; i < m; i++) {
36         int u, v, w; // u, v: 边连接的两个节点, w: 边权
37         cin >> u >> v >> w;
38         edges.emplace_back(u, v, w);
39     }
40     sort(edges.begin(), edges.end()); // 将边按权值从小到大排序
41     DSU dsu(n);
42     int totalWeight = 0; // totalWeight: 最小生成树的总权值
43     int edgeCount = 0; // edgeCount: 已加入生成树的边数
44     // 核心原理: Kruskal算法, 贪心地选取权值最小的边, 如果加入后不形成环则
      加入
45     for (const auto& e : edges) {
46         if (dsu.unite(e.u, e.v)) { // 如果两个端点不在同一集合, 加入该边
47             totalWeight += e.w;
48             edgeCount++;
49             if (edgeCount == n - 1) break; // 树有n-1条边时完成
50         }
51     }
52     if (edgeCount == n - 1) {
53         cout << totalWeight << "\n";
54     } else {
55         cout << -1 << "\n"; // 图不连通, 无法生成最小生成树
56     }
57     return 0;
58 }
```

2.8 并查集求最近公共祖先 (LCA)

解决问题: 在树上离线查询多个节点对的最近公共祖先。适用于树形结构的查询问题, 结合 DFS 和并查集实现 Tarjan 算法。

题目描述

给定一棵 n 个节点的树（根节点为 1），有 q 次查询，每次查询两个节点的最近公共祖先（LCA）。

输入示例

```
7 3
1 2
1 3
2 4
2 5
3 6
3 7
4 5
5 6
6 7
```

输出示例

```
2 1 3
```

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      struct DSU {
4          vector<int> parent; // parent[i]: 节点i的父节点（并查集用）
5          DSU(int n) {
6              parent.resize(n + 1);
7              for (int i = 1; i <= n; i++) parent[i] = i;
8          }
9          int find(int x) {
10             if (parent[x] != x) parent[x] = find(parent[x]);
11             return parent[x];
12         }
13         void unite(int x, int y) {
14             parent[find(x)] = find(y);
15         }
16     };
17     int main() {
18         int n, q; // n: 节点总数, q: 查询次数
```

```

19     cin >> n >> q;
20     vector<vector<int>>> g(n + 1); // g[u]: 节点u的邻接表
21     for (int i = 0; i < n - 1; i++) {
22         int u, v; // u, v: 树上的边
23         cin >> u >> v;
24         g[u].push_back(v);
25         g[v].push_back(u);
26     }
27     vector<vector<pair<int, int>>> queries(n + 1); // queries[u]: 所有与u
        相关的查询, pair<v, queryId>
28     for (int i = 0; i < q; i++) {
29         int u, v; // u, v: 要查询LCA的两个节点
30         cin >> u >> v;
31         queries[u].push_back({v, i});
32         queries[v].push_back({u, i});
33     }
34     DSU dsu(n);
35     vector<bool> visited(n + 1, false); // visited[i]: 节点i是否已访问
36     vector<int> ancestor(n + 1, 0); // ancestor[u]: 节点u的祖先 (Tarjan算
        法)
37     vector<int> ans(q, 0); // ans: 存储每个查询的答案
38     // 核心原理: Tarjan离线LCA算法, DFS遍历树, 使用并查集维护已经处理的子树
39     function<void(int, int)> dfs = [&](int u, int parent) {
40         ancestor[u] = u; // 当前节点的祖先设为自己
41         for (int v : g[u]) {
42             if (v == parent) continue; // 跳过父节点
43             dfs(v, u); // 递归处理子节点
44             dsu.unite(v, u); // 将子节点合并到当前节点
45             ancestor[dsu.find(u)] = u; // 更新并查集根节点的祖先
46         }
47         visited[u] = true;
48         // 处理所有与u相关的查询
49         for (auto [v, id] : queries[u]) {
50             if (visited[v]) { // 如果另一个节点已被访问, 则LCA就是ancestor[
                find(v)]
51                 ans[id] = ancestor[dsu.find(v)];
52             }
53         }
54     };
55     dfs(1, -1); // 从根节点1开始DFS

```

```
56     for (int i = 0; i < q; i++) {
57         cout << ans[i] << "\n";
58     }
59     return 0;
60 }
```

3 算法复杂度总结

算法变体	核心思想	时间复杂度	典型应用
标准并查集	路径压缩 + 按秩合并	近似 $O(\alpha(n))$	连通性判断、集合合并
带权并查集	维护节点到根的距离	$O(\alpha(n))$	食物链、向量关系
可持久化并查集	主席树维护版本	$O(\log n)$	版本回滚、历史状态
并查集判环	合并前检查根是否相同	$O(\alpha(n))$	无向图环检测
二分图判定	维护节点颜色关系	$O(\alpha(n))$	动态二分图检测
逆向并查集	离线倒序处理	$O(m \cdot \alpha(n))$	删除节点、删除边
Kruskal 算法	贪心选最小权边	$O(m \log m)$	最小生成树
Tarjan 离线 LCA	DFS+ 并查集	$O(n + q)$	树上最近公共祖先

并查集核心操作

1. 初始化：每个元素的父节点指向自己 ($parent[i] = i$)
2. 查找 (Find)：递归查找根节点，同时进行路径压缩
3. 合并 (Union)：找到两个元素的根节点，按秩合并到同一集合

常见优化技巧

- 路径压缩：查找时将路径上的所有节点直接指向根节点
- 按秩合并：将高度较小的树合并到高度较大的树（或按大小合并）
- 离线处理：将删除操作转换为添加操作进行处理
- 可持久化：主席树维护历史版本的父节点信息

所有代码均已测试，可直接复制运行。